Railway Ticket Booking System

Report for Mini-Project using SE perspective

6th semester, B.Tech(2020-24)

in

Computer Science and Engineering
(Tezpur University)

Grateful for the invaluable guidance and mentorship provided

by

Dr. Shobhanjana Kalita
Assistant Professer
Department of Computer Science and Engineering
(Tezpur University)

Submitted By

Rahul Kumar Sah
CSB20045
Department of Computer Science and Engineering

CONTENTS

1. Executive Summary
1.1 Introduction
1.2 Project Objectives

2. Introduction
2.1 Background and Context
2.2 Problem Statement
2.3 Project Scope
2.4 Project Deliverables
2.5 Stakeholders

3. Feasibility Study
3.1 Technical Feasibility
3.2 Economic Feasibility
3.3 Operational Feasibility

4. Software Requirements Specifications
4.1 Introduction
4.2 Overall Description
4.3 Functional Requirements
4.4 Non-Functional Requirements
4.5 Use Cases
4.6 System Features
4.7 System Constraints
4.8 Assumptions and Dependencies
4.9 System Testing

5. System Design
5.1 Architecture Design
5.2 Database Design
5.3 Context Diagram
5.4 Data flow diagram
5.5 Algorithm esign
5.6 System maintenance
5.7 System Development Approach

6. Implementation
6.1 System Development Platform
6.2 User Interfaces
6.3 Database

7. Data Dictionay

8. Conclusion
8.1 Outcomes
8.2 Future Refinements
8.3 Learnings

BIBLIOGRAPHY

1. Executive Summary

1.1 Introduction

The railway ticket booking system is an essential component of modern railway operations , facilitating efficient and convenient booking of train tickets for passengers. This project streamlines the ticket reservation process , allowing users to search for available trains, Select trains on the basis of their choices and book tickets for that train, users can book multiple tickets, ticket booking is made secure using OTP based verification, allowing users to print tickets, users recieve ticket information on their emails after confirmation of ticket, users can cancel tickets , check pnr status for a particular ticket. Admin section is added for management of trains , updating train information , adding stations , adding trains etc.

1.2 Project Objectives

. **Improve user experience**

Enhance the overall user experience by providing a user friendly interface , intuitive navigation , and efficient ticket booking process , providing booking and cancellation services.

. **Provide real-time information**

Display accurate and up-to-date train schedules , seat availability , seat availablity , fares and other relevant information to help users make informed decisions, real-time train and ticket status.

. Secure ticket booking

Ensuring secure and hassle free ticket booking using OTP based verification during ticket booking process.

. **Hassle free management**

Providing GUI for smooth and hassle free management on the admin side, for example train information updation, train addition and management of users and trains and all other entities involved.

. Support Multiple Platforms

Ensure compatibility with various platforms and devices, since it is a web application , there is no problem with running on multiple platforms.

. Enable integration

Allow seamless integration with other systems or services such as reward programs , passenger information systems or third party travel agencies, to enhance and increase convenience and reach to more numbers of passengeres

. Scalability and Maintainability

Design the system to be scalable, accommodating future growth and increasing demands. Additionally, prioritize maintainability to facilitate system updates, bug fixes, and enhancements over time.

2. Introduction

2.1 Background and Context

Modern railway operations require a modern, secure and efficient ticket reservation system. Enhancing the existing ticket booking process is crucial to meet the demand for a user friendly and streamlined reservation process. Leveraging technology advancements such as online ticket booking and mobile application based ticket booking system, integration with other railway systems and third party agencies to cater to a large group of customers are key goals. Developing a robust and real- time reservation system serves the purpose of railways vision of providing a seamless travel experience for millions of passengers while enhancing efficiency and revenue generation.

2.2 Problem statement

The existing ticket booking system faces several challenges , including long queues , manual errors , limited access to ticket information , and difficulties in managing ticket inventory.

These issues lead to inconvenience for passengers and inefficiencies in the overall booking process. Additionally , the traditional methods are unable to cope with the increasing demand and advancements in technology. Therefore , there is a need to develop a modernized and efficient train ticket booking system that addresses these challenges and provides a seamless and user friendly experience for passengers.

2.3 Problem Scope

The train ticket booking system project aims to design and implement a comprehensive solution that tackles the identified problems . The scope includes developing a user friendly interface for booking tickets, implementing automated processes to reduce manual errors , integrating real-time information on train schedules and seat availability, and providing secure payment options. The system will also focus on efficient ticket management, including features for cancellations and refunds. Morever , the project aims to ensure scalability and compatibility across multiple platforms to accomodate future growth and technological advancements. By addressing these issues, the project seeks to streamline the ticket booking process and enhance the overall experience for passengers.

2.4 Project Deliverables

- Developed software application for online ticket booking.
- User documentation, including user manuals and guides.
- System documentation, detailing the architecture, design, and implementation.
- Integration with payment gateways for secure online transactions.
- Real-time information display for train schedules, seat availability, and fares.
- Reporting and analytics capabilities for data-driven decision-making.
- Scalable and maintainable system infrastructure to accommodate future growth and updates.

2.5 Stakeholders

The key stakeholders involved are :

- Passengers: The primary users of the system, seeking a convenient and hassle-free ticket booking experience.
- Railway Operators: The railway authorities responsible for managing and operating the ticket booking system.
- Administrators: The system administrators responsible for maintaining and monitoring the system's performance and security.

- Technical Team: The team involved in the development, implementation, and maintenance of the system.
- Payment Gateway Providers: Companies providing secure payment processing services for online transactions.
- Third-Party Travel Agencies: Travel agencies that may integrate with the system to offer ticket booking services to their customers.
- Regulatory Authorities: Government bodies or organizations responsible for overseeing and regulating railway operations.

These stakeholders collectively contribute to the successful implementation and utilization of the Train Ticket Booking System.

3. Feasibility Study

3.1 Technical Feasibility:

- Availability of technological resources and infrastructure required for developing and implementing the train ticket booking system.

- Compatibility of the system with existing hardware and software platforms used by the Indian Railways.

- Ability to handle a large volume of concurrent users and transactions without compromising system performance.

- Integration capabilities with other systems and services, such as payment gateways, passenger information systems, and seat availability databases.

- Scalability of the system to accommodate future growth and increased demand.

- Adequate security measures to protect user data and ensure privacy during online transactions.

3.2 Economic Feasibility:

- Cost-benefit analysis to determine the financial viabilityof developing and maintaining the train ticket booking system.

- Evaluation of the costs associated with system development, infrastructure setup, software licensing, and ongoing maintenance.

- Assessment of potential cost savings or revenue generation opportunities through increased efficiency and reduced manual ticketing processes.

- Consideration of the return on investment (ROI) and payback period for the implementation of the system.

- Analysis of potential revenue streams, such as service charges, booking fees, or advertising opportunities.

- Comparison of the overall cost of system implementation with the expected benefits and savings.

3.3 Operational Feasibility:

- Analysis of the system's alignment with existing operational processes and workflows within the Indian Railways.

- Evaluation of the ease of system implementation and integration with existing ticketing and reservation procedures.

- Assessment of the impact on personnel training and readiness for adopting and operating the new system.

- Examination of any potential disruptions to current operations during the system implementation phase.

- Consideration of user acceptance and willingness to transition from traditional ticketing methods to the new system.

- Evaluation of the system's ability to improve operational efficiency, reduce manual errors, and enhance customer satisfaction.

4. Software Requirements Specifications

4.1. Introduction

. Purpose of the document

This document's goal is to give a thorough overview of the railway ticket booking system. It details the interactions between users and the system , the functional and non-functional requirements of the system, and the overall scope of the software. The development team, participants and any other parties participating in the project can refer to this document as a reference.

. Scope of the system

The railway ticket booking system is made to make it easier for users to see available trains for any route , register , book tickets , monitor their ticket status , cancel their tickets and for admins to add trains , classes , stations , update train information, station and class

information and various other management options for smooth handling of trains and ticket booking. ReactJs , HTML , Tailwind-Css are used for user-interface , while NodeJs , mongoDB , Express are used for server-side processing of user and admin activities.

. Definitions , Acronyms and Abbreviations

. SRS : Software Requirements Specifications

. ReactJs : JavaScript framework used for building user interfaces

. HTML : Markup language required for building web applications

. Tailwind-Css : Css framework used for styling web user interfaces using all the pre-defined css classes

. Nodejs : Server side JavaScript framework/library used for creating apis

. Express : Tool for server side programming alongwith NodeJs

. MongoDB : Database connectivity , non-relational database service provided for developing applications , used in Server side for handling all the data

4.2. Overall Description

. External interfaces and requirements

. User interfaces

User interfaces for the railway ticket booking system application will be as follows:

- Login interface : Students and administrators can log in to the system using this interface by entering their username and passwords.

- User Profile page : Upon successful registration users are redirected to their profile page, which contains user information and various options like if the user has any unfinished booking then user has option to continue that booking , start new booking.

- Train Search : Users can search for trains from any station to any other station, suggestions are displayed for stations, users can search for a specific train using train number or train name as all matching train names and numbers are displayed as the user types , users can search trains for some specific class, search for trains on a specific date.

- Train List : List of trains is displayed as per queries by users. This page displays all the trains available for user query, along with name , number of the trains , start and ending stations, available classes , running days of the train.

- Train information : Information about selected train is shown along with fair details and time-table for the train. Users can select any ticket class of their choice and move to reservation details page.

- Reservation Details : Reservation details like date of travel , details of all the passengers which will be travelling are entered.

- Reservation Confirmation : Details of reservation are shown, edit option is available in case user wants to edit details of passengers,OTP verification is done before moving for payment section, after verification of OTP, payment is done, upon confirmation reservation details is sent to the user registered email.

- Print Ticket : Options for printing all the tickets are available for user to print all the tickets once or one by one.

- Admin Section : Admin section has interfaces for adding trains , adding classes , and adding stations, also updating information about trains, stations , classes.

- Hardware Interfaces : There are no particular hardware interface specifications needed for the railway ticket booking system application. This is a web application which can be used from any device.

-Software Interfaces : There are no particular software interface specifications needed for the railway ticket booking system application. This is a web application which can be used from any device.

4.3. Fuctional Requirements

4.3.1 User functionality

User Functionality:

1. Ticket Booking

- Users should be able to search for available trains and book tickets for their desired journey.
- They should be able to select the source and destination stations, travel date, class, and number of passengers.
- The system should provide real-time availability and suggest alternative options if the desired train is not available.

2. Train Searching

- Users should be able to search for trains based on criteria such as source and destination stations, travel date, and class.
- The system should display a list of matching trains along with their schedules and availability.

3. User Login and Registration

- Users should be able to create an account or log in using their credentials.
- Registration should require basic information like name, email, and password.
- The system should provide password reset functionality in case the user forgets their password.

4. Profile Access and Update

- Users should have access to their profile where they can view and update their personal information.
- They should be able to manage contact details, preferences, and saved passenger information.

5. PNR Status Checking

- Users should be able to check the status of their booked tickets using the PNR (Passenger Name Record) number.
- The system should provide real-time updates on the ticket status, including confirmation, waiting list, or cancellation.

6. Ticket Cancellation

- Users should have the ability to cancel their booked tickets.
- The system should provide a user-friendly interface to initiate and confirm ticket cancellations.
- Refund calculations and policies should be applied as per the specified rules.

7. History of Reservations and Ticket Cancellation

- Users should be able to view their booking history, including past and upcoming journeys.
- They should have the option to cancel tickets from their reservation history.

8. Email and OTP Verification

- The system should send email notifications to users regarding ticket confirmation, cancellation, and updates.
- Email verification should be implemented during registration and for critical actions like password reset.

9. Reservation Details Edit

- Users should be able to modify certain details of their existing reservations, such as passenger names, travel date, or class.
- The system should validate the changes and provide appropriate notifications.

Admin Functionality:

1. Add Train

- Admins should have the ability to add new trains to the system.
- They should enter train details like name, number, source, destination, and schedule information.

2. Add Classes

- Admins should be able to define and add different classes of travel (e.g., First Class, Second Class, Sleeper, etc.).

- They should specify class-specific amenities, facilities, and fare structures.

3. Add Stations

- Admins should have the authority to add new stations to the system.

- They should provide station details such as name, code, and geographical information.

4. Update Trains

- Admins should be able to edit and update train information, including train details, schedules, and availability.

5. Update Classes

- Admins should have the ability to modify class-specific amenities, facilities, and fare structures.

6. Update Stations

- Admins should be able to update station details such as names, codes, and geographical information.

7. Manage Tickets

- Admins should have access to ticket management functionality.

- They should be able to view, search, and manage all booked tickets.

- The system should allow them to generate reports, statistics, and handle exceptional cases.

8. Update Train Status

- Admins should be able to update the status of trains, such as delayed, canceled, or rescheduled.

- Users should be notified of any changes in the train status.

4.4 Non-functional Requirements :

1. Performance:

- The system should be able to handle a high volume of concurrent users without significant performance degradation.
- Response times for ticket booking, train search, and other critical operations should be fast to provide a seamless user experience.
- The system should have efficient database indexing and query optimization to ensure quick retrieval of data.

2. Scalability:

- The system should be scalable to accommodate increasing user demand and growing data.
- It should be able to handle a larger number of trains, stations, and user profiles without sacrificing performance.
- Scalability should be achieved through appropriate hardware infrastructure and system architecture design.

3. Security:

- User authentication and authorization should be implemented to ensure secure access to the system.
- Sensitive user data, such as personal information and payment details, should be encrypted and stored securely.
- The system should have mechanisms to prevent unauthorized access, data breaches, and protect against common web vulnerabilities.

4. Reliability:

- The system should be highly reliable, minimizing the occurrence of errors, crashes, or downtime.

- It should have backup and recovery mechanisms to ensure data integrity and availability in case of system failures.

- Proper error handling and graceful degradation should be implemented to handle unexpected scenarios and provide a smooth user experience.

5. Availability:

- The system should be available to users round the clock, allowing ticket booking and other operations at any time.

- Scheduled maintenance activities should be planned during off-peak hours to minimize user impact.

- Redundancy and failover mechanisms should be in place to ensure continuous availability in case of hardware or network failures.

6. Usability:

- The user interface should be intuitive, user-friendly, and visually appealing.

- Clear and informative error messages should be provided to guide users in case of input errors or system failures.

- The system should support multiple devices and screen sizes, providing a responsive design for mobile and desktop users.

7. Compliance:

- The system should comply with relevant data protection and privacy regulations, such as GDPR or local data protection laws.

- Payment processing should adhere to industry-standard security protocols and PCI-DSS compliance.

- Accessibility standards should be followed to ensure the system is usable by individuals with disabilities.

8. Performance Testing:

- The system should undergo rigorous performance testing to identify bottlenecks, load handling capacity, and optimize resource utilization.

- Load testing should be performed to simulate heavy user traffic and assess system performance under peak loads.

4.5. Use Cases

4.5.1 User Use Cases :

1. Ticket Booking:

Actors: Users

Description:

- Users search for available trains based on source, destination, date, class, and passenger details.
- The system checks the availability of seats for the selected train and class.
- Users select the desired train and proceed to enter passenger details.
- The system validates the input, calculates the fare, and generates a unique PNR (Passenger Name Record) number.
- Users receive a booking confirmation with the ticket details and payment information.

Flow of Events:

1. User enters source, destination, date, class, and passenger details.
2. System retrieves matching trains and displays the available options.
3. User selects a train and proceeds to enter passenger details.
4. System validates the input and calculates the fare.
5. User confirms the booking and proceeds to make the payment.
6. System generates a unique PNR number and provides a booking confirmation to the user.

2. Train Searching:

Actors: Users

Description:

- Users search for trains based on source, destination, date, and class.
- The system retrieves a list of matching trains with their schedules and availability.

- Users can view train details, including departure and arrival times, duration, and intermediate stops.

- Users can filter and sort the search results based on preferences such as travel time or fare.

- Users select the desired train for further actions, such as ticket booking.

Flow of Events:

1. User enters the source, destination, date, and class for train search.

2. System retrieves matching trains and displays the search results.

3. User views train details, including schedules and availability.

4. User applies filters or sorts the results based on preferences.

5. User selects a specific train for further actions, such as ticket booking.

3. User Registration:

Actors: New Users

Description:

- New users create an account by providing their basic information, such as name, email, and password.

- The system validates the input and checks for email uniqueness.

- Users receive a verification email to confirm their email address.

- Users click on the verification link or enter the received OTP (One-Time Password) to activate their account.

- Upon successful verification, users can log in to the system.

Flow of Events:

1. User clicks on the registration link or navigates to the registration page.

2. User enters their basic information, including name, email, and password.

3. System validates the input and checks for email uniqueness.

4. User receives a verification email with a link or OTP.

5. User clicks on the verification link or enters the OTP to activate their account.

6. System confirms the activation and notifies the user.

4. User Login:

Actors: Registered Users

Description:

- Registered users log in to the system using their email and password.
- The system verifies the user's credentials and grants access to their account.
- Users may need to complete additional security measures like captcha verification or two-factor authentication.
- Upon successful login, users gain access to their profile, bookings, and other functionalities.
- Invalid login attempts or forgotten passwords trigger appropriate error messages or password reset options.

Flow of Events:

1. User navigates to the login page and enters their registered email and password.
2. System verifies the credentials and performs additional security checks if required.
3. If the login is successful, the system grants access to the user's account and associated functionalities.
4. In case of invalid login attempts, the system displays appropriate error messages.
5. Users may have options for password reset or account recovery if they forget their login credentials.

5. Profile Access:

Actors: Users

Description:

- Users can access their profile to view and manage personal information, contact details, and preferences.
- The profile may display the user's name, email, phone number, address, and other relevant information.
- Users can also view and manage their saved passenger information, such as names, ages, and seat preferences.
- The profile section provides a centralized location for users to review and update their details as needed.

- Users may have options to view their booking history, upcoming trips, and other relevant information.

Flow of Events:

1. User navigates to the profile section of the application.

2. System displays the user's personal information, contact details, and preferences.

3. User can view and update their details, such as name, email, phone number, address, etc.

4. User can access saved passenger information and modify it as needed.

5. Users may have options to view their booking history, upcoming trips, or additional information related to their profile.

6. Profile Update:

Actors: Users

Description:

- Users can update their profile information, such as contact details, preferences, and saved passenger information.

- They have the ability to modify their name, email, phone number, address, and other relevant details.

- Users can also add, edit, or delete passenger information associated with their account.

- The system validates the updated information and ensures data consistency.

- The updated profile information is reflected in future bookings and profile access.

Flow of Events:

1. User navigates to the profile section of the application.

2. User selects the option to update their profile information.

3. User modifies the desired fields, such as contact details, preferences, or saved passenger information.

4. System validates the updated information and checks for any conflicts or errors.

5. If the updated information is valid, the system applies the changes and updates the user's profile.

7. PNR Status Checking:

Actors: Users

Description:

- Users can check the status of their booked tickets by entering the PNR (Passenger Name Record) number.

- They enter the PNR number and submit the request to retrieve the booking details.

- The system retrieves the booking information associated with the provided PNR number.

- Users can view the status of their tickets, including whether they are confirmed, on the waiting list, or canceled.

- The system provides real-time updates on the ticket status.

Flow of Events:

1. User navigates to the PNR status checking section or enters the PNR number on the homepage.

2. User enters the PNR number associated with their booked ticket.

3. System retrieves the booking details corresponding to the provided PNR number.

4. User views the status of their tickets, such as confirmed, waiting list, or canceled.

5. System provides real-time updates on any changes to the ticket status, if applicable.

8. Ticket Cancellation:

Actors: Users

Description:

- Users can cancel their booked tickets if needed.

- They select the ticket(s) they wish to cancel from their booking history.

- The system validates the cancellation request and calculates any applicable refund.

- Users confirm the cancellation, and the system updates the booking status and refund amount accordingly.

- The canceled tickets are removed from the user's booking history.

Flow of Events:

1. User navigates to their booking history or ticket details section.

2. User selects the ticket(s) they wish to cancel.

3. System validates the cancellation request and calculates the refund (if applicable) based on cancellation policies.

4. User confirms the cancellation and proceeds with the process.

5. System updates the booking status to "canceled" and refunds the applicable amount to the user's account.

9. History of Reservations and Ticket Cancellation:

Actors: Users

Description:

- Users can access their booking history to view past and upcoming reservations.

- They can select specific bookings from the history and cancel the tickets if needed.

- The system displays relevant details such as train name, journey dates, class, and ticket status.

- Users can navigate through their reservation history and perform cancellation for eligible tickets.

- The system updates the booking records and refunds the applicable amount, if applicable.

Flow of Events:

1. User navigates to the booking history section of their profile.

2. System displays the user's past and upcoming reservations.

3. User selects a specific booking

from the history to view the details.

4. User can choose to cancel eligible tickets within the cancellation window.

5. System updates the booking records, changes the ticket status to "canceled," and processes any applicable refund.

10. Email and OTP Verification:

Actors: Users

Description:

- Users may receive emails or OTPs (One-Time Passwords) for various purposes, such as registration, ticket booking, or password reset.

- The system sends emails or OTPs to the user's registered email address or phone number.

- Users enter the received OTP or follow the instructions provided in the email for verification.

- Verification ensures the authenticity and security of user actions, such as account creation or critical transactions.

- The system verifies the entered OTP or completion of the required actions before allowing further user interactions.

Flow of Events:

1. User initiates an action that requires verification, such as registration, booking, or password reset.

2. System generates and sends an email or OTP to the user's registered email address or phone number.

3. User receives the email or OTP and enters the provided code or follows the instructions in the email.

4. System verifies the entered OTP or completion of the required actions.

5. If the verification is successful, the system allows the user to proceed with the desired action.

4.5.2 Admin Use Cases :

1. Add Train:

Actors: Admins

Description:

- Admins have the ability to add new trains to the system.

- They enter details such as train name, number, source, destination, and schedules.

- Admins can provide additional information like train type, seating capacity, and facilities.

- The system validates the input and adds the train to the database.

- The newly added train becomes available for users to search and book tickets.

Flow of Events:

1. Admin navigates to the "Add Train" section of the admin panel.

2. Admin enters the train details, including name, number, source, destination, and schedules.

3. Admin provides additional information like train type, seating capacity, and facilities.

4. System validates the input and checks for any conflicts or errors.

5. If the input is valid, the system adds the train to the database and makes it available for users.

2. Add Classes:

Actors: Admins

Description:

- Admins can define and add different travel classes to the system.

- They specify the amenities, facilities, and fare structures for each class.

- Admins may assign unique identifiers or codes to distinguish between classes.

- The system validates the input and adds the classes to the system.

- Users can then select their preferred class while booking tickets.

Flow of Events:

1. Admin navigates to the "Add Classes" section of the admin panel.

2. Admin defines the class details, including name, amenities, facilities, and fare structures.

3. Admin may assign unique identifiers or codes to each class for easy identification.

4. System validates the input, checks for any conflicts, and ensures consistency.

5. If the input is valid, the system adds the classes to the system, making them available for users during ticket booking.

3. Add Stations:

Actors: Admins

Description:

- Admins have the ability to add new stations to the system.

- They provide details such as station name, code, and geographical information.

- Admins may also include additional information like station facilities and nearby attractions.

- The system validates the input and adds the stations to the database.

- The newly added stations become part of the available station list for users.

Flow of Events:

1. Admin navigates to the "Add Stations" section of the admin panel.

2. Admin enters the station details, including name, code, and geographical information.

3. Admin may include additional information like station facilities and nearby attractions.

4. System validates the input and checks for any conflicts or errors.

5. If the input is valid, the system adds the stations to the database, making them available for users to select as source or destination stations.

4. Update Trains:

Actors: Admins

Description:

- Admins can edit and update train information in the system.

- They have the ability to modify details such as train name, number, source, destination, and schedules.

- Admins can also update additional information like train type, seating capacity, and facilities.

- The system validates the updated information and ensures consistency across the system.

- The updated train information reflects the changes for users during train searching and ticket booking.

Flow of Events:

1. Admin navigates to the "Update Trains" section of the admin panel.

2. Admin selects the train to be updated from the list of available trains.

3. Admin modifies the train details, including name, number, source, destination, and schedules.

4. Admin can also update additional information like train type, seating capacity, and facilities.

5. System validates the updated information, checks for any conflicts, and ensures consistency.

6. If the updated information is valid, the system applies the changes and updates the train information in the database.

5. Update Classes:

Actors: Admins

Description:

- Admins have the ability to modify travel class details in the system.

- They can update class names, amenities, facilities, and fare structures.

- Admins may also adjust seating capacity or make changes to existing class configurations.

- The system validates the updated information and ensures consistency across the system.

- The updated class details are reflected during ticket booking for users to select their preferred class.

Flow of Events:

1. Admin navigates to the "Update Classes" section of the admin panel.

2. Admin selects the class to be updated from the list of available classes.

3. Admin modifies the class details, including name, amenities, facilities, and fare structures.

4. Admin may also adjust seating capacity or make changes to existing class configurations.

5. System validates the updated information, checks for any conflicts, and ensures consistency.

6. If the updated information is valid, the system applies the changes and updates the class details in the system.

6. Update Stations:

Actors: Admins

Description:

- Admins can update the details of existing stations in the system.

- They have the ability to modify station names, codes, geographical information, and additional details.

- The system validates the updated information and ensures consistency.

- The updated station information is reflected in train searching, ticket booking, and other relevant functionalities.

Flow of Events:

1. Admin navigates to the "Update Stations" section of the admin panel.

2. Admin selects the station to be updated from the list of existing stations.

3. Admin modifies the station details, such as name, code, geographical information, and additional details.

4. System validates the updated information, checks for any conflicts or errors, and ensures consistency.

5. If the updated information is valid, the system applies the changes and updates the station details in the system.

7. Manage Tickets:

Actors: Admins

Description:

- Admins have the ability to manage and view ticket details for bookings made by users.

- They can search for bookings using criteria like PNR, passenger name, or journey details.

- Admins can view and modify ticket information, such as passenger names, journey dates, seat assignments, and class.

- They can also perform actions like ticket cancellation, rescheduling, or generating reports.

- The system updates the ticket records based on the actions performed by admins.

Flow of Events:

1. Admin navigates to the "Manage Tickets" section of the admin panel.

2. Admin searches for bookings using criteria like PNR, passenger name, or journey details.

3. System retrieves the relevant ticket details and displays them to the admin.

4. Admin can view and modify ticket information, such as passenger names, journey dates, seat assignments, and class.

5. Admin performs actions like ticket cancellation, rescheduling, or generating reports.

6. System updates the ticket records accordingly based on the actions performed by admins.

8. Update Train Status:

Actors: Admins

Description:

- Admins can update the status of trains, such as "on time," "delayed," or "canceled."
- They can input the train number or select the train from the list to update the status.
- The system validates the updated status and reflects it in train searching and ticket booking functionalities.
- Users can view the updated train status when searching for trains or checking their booking details.
- The system may send notifications to affected passengers in case of train cancellations or delays.

Flow of Events:

1. Admin navigates to the "Update Train Status" section of the admin panel.
2. Admin inputs the train number or selects the train from the list.
3. Admin updates the train status to "on time," "delayed," or "canceled."
4. System validates the updated status and ensures data consistency.
5. The updated train status is reflected in train searching and ticket booking functionalities for users to view.

9. Add Additional Features (Custom Feature):

Actors: Admins

Description:

- Admins have the ability to add custom features or functionalities to the railway ticket booking system.
- These additional features can enhance user experience or provide unique services.
- Admins define the scope and specifications of the custom feature.
- The system incorporates the additional feature into the existing system, ensuring compatibility and usability.

- Users can access and utilize the custom feature as part of their ticket booking journey.

Flow of Events:

1. Admin identifies a need for an additional feature or functionality in the railway ticket booking system.

2. Admin defines the scope, specifications, and desired outcome of the custom feature.

3. System architects and developers analyze the feasibility and impact of the additional feature.

4. Developers incorporate the custom feature into the existing system, ensuring compatibility and usability.

5. Users can access and utilize the custom feature as part of their ticket booking journey.

10. Generate Reports:

Actors: Admins

Description:

- Admins can generate reports based on various parameters and data within the railway ticket booking system.

- They can generate reports on ticket sales, revenue, passenger statistics, train occupancy, and more.

- Admins can specify the time period, filters, and format of the generated reports.

- The system retrieves the relevant data and generates comprehensive reports for analysis and decision-making.

- Admins can export or share the generated reports in different formats for further use.

Flow of Events:

1. Admin navigates to the "Generate Reports" section of the admin panel.

2. Admin selects the type of report to be generated, such as ticket sales, revenue, or passenger statistics.

3. Admin specifies the time period, filters, and format of the generated report.

4. System retrieves the relevant data based on the selected parameters.

5. System generates a comprehensive report with the requested information and presents it to the admin.

6. Admin can export or share the generated report in different formats, such as PDF or Excel, for further analysis and use.

4.6 System Features

1. User Functionality:

- Ticket Booking
  - Search for trains
  - Select train and class
  - Enter passenger details
  - Make payment
  - Receive booking confirmation

- Train Searching
  - Search for trains based on source and destination
  - View train details, schedules, and available classes

- User Login Registration
  - Register a new user account
  - Log in to an existing account
  - Authenticate user credentials

- Profile Access
  - View user profile information
  - Access saved preferences and passenger details
  - Edit and update profile information

- PNR Status Checking
  - Enter PNR number
  - Retrieve booking details
  - View ticket status and journey information

- Ticket Cancellation
  - Access booking history
  - Select ticket(s) for cancellation
  - Calculate refund amount
  - Confirm cancellation and update booking status

- History of Reservations with Ticket Cancellation Option
  - View past and upcoming reservations
  - Select booking for cancellation
  - Process cancellation and update records
  - Refund applicable amount, if any

- Email and OTP Verification: Send verification emails and OTPs , Receive and validate entered OTPs, Verify user actions and ensure security.
- Reservation Details Edit: Edit passenger details in a booked ticket , Modify journey dates or class, if applicable , Update ticket information and records

2. Admin Functionality:
   - Add Train
     - Enter train details: name, number, source, destination
     - Define train schedules and stops
     - Set seating capacity and facilities

   - Add Classes
     - Create new travel classes

  - Define class amenities and fare structures
  - Specify seating capacity and configurations

- Add Stations
  - Add new station details: name, code, location
  - Update geographical information

- Update Trains
  - Modify train details: name, number, source, destination
  - Update train schedules and stops
  - Adjust seating capacity and facilities

- Update Classes
  - Edit class details: name, amenities, fare structures
  - Modify seating capacity and configurations

- Update Stations
  - Update existing station details: name, code, location
  - Modify geographical information

- Manage Tickets
  - Search and retrieve ticket details
  - View passenger information and journey details
  - Modify ticket information (e.g., passenger names, journey dates)
  - Perform actions like cancellation, rescheduling, and generating reports

- Update Train Status
  - Input train number or select train
  - Update train status (e.g., on time, delayed, canceled)
  - Reflect updated status in train searching and ticket booking

- Add Additional Features (Custom Feature)
  - Define scope and specifications of additional feature
  - Incorporate custom feature into the system
  - Ensure compatibility and usability for users

- Generate Reports
  - Select report type and parameters
  - Retrieve relevant data from the system
  - Generate comprehensive reports
  - Export or share reports in different formats (e.g., PDF, Excel)

4.7 System Constraints

1. User Functionality Constraints:

- Ticket Booking: Constraints related to ticket booking limits, such as maximum number of tickets per transaction or per user. Constraints related to payment options, such as supported payment methods or integration with third-party payment gateways.

- Train Searching: Constraints related to search speed and performance, ensuring quick retrieval of train information based on user queries. Constraints related to search filters and parameters, allowing users to refine their search results effectively.

- User Login and Registration: Constraints related to user authentication and security measures, including password complexity requirements and secure storage of user credentials. Constraints related to unique identification of users, such as email verification or mobile number verification.

- Profile Access and Update: Constraints related to data privacy and access control, ensuring that users can only access and update their own profile information.Constraints related to data validation, enforcing rules for data formats, input length, or character limits.

- PNR Status Checking: Constraints related to real-time data updates and synchronization with the ticketing system to provide accurate PNR status information.

- Ticket Cancellation: Constraints related to cancellation policies, including time limits for cancellations and applicable refund rules. Constraints related to ticket status updates and synchronization with the booking system after cancellation.

- History of Reservations with Ticket Cancellation: Constraints related to data retention, ensuring the availability of historical reservation details for a specified period Constraints related to refund processing, adhering to refund policies and processing timelines.

- Email and OTP Verification: Constraints related to email delivery and integration with email service providers to ensure timely delivery of verification emails. Constraints related to OTP generation and validation, including OTP validity duration and security measures.

2. Admin Functionality Constraints:

- Add Train, Classes, and Stations: Constraints related to data validation and integrity, ensuring unique train numbers, class names, and station codes. Constraints related to data entry forms, enforcing required fields and appropriate data formats.

- Update Trains and Classes: Constraints related to data synchronization and consistency, ensuring that train and class updates are reflected consistently throughout the system.

- Update Stations: Constraints related to geographical data accuracy, validating the correctness of station coordinates and other geographical information.

- Manage Tickets: Constraints related to data access and security, allowing admins to view and modify tickets only within their authorized scope. Constraints related to ticket modification limits, such as restrictions on changing passenger names or journey dates.

- Update Train Status: Constraints related to real-time train status updates, integrating with external data sources or APIs to provide accurate train status information.

4.8 Assumptions and Dependencies :

The railway ticket booking system is an web application the dependencies are any modern browser and stable internet connection.

4.9 System Testing

System tesing is a crucial phase in the development lifecycle of any project.

It involves testing the entire system as a whole to ensure that all the components, functionalities ,and interactions work together seamlessly. The goal of system testing is to validate the system's compliance with requirements

4.9.1 Types of System Testing:

Functional Testing: This testing phase involves validating each user functionality, such as ticket booking, train searching, profile access, and cancellation, against the specified requirements. Test cases are designed to cover different scenarios, including positive and negative test cases, boundary values, and error handling. The aim is to verify that all the user functionalities are working as expected and producing accurate results.

Performance Testing: Performance testing is essential to evaluate the system's performance under different load conditions. It includes load testing to determine how the system behaves when multiple users are accessing it simultaneously, stress testing to identify system limits and bottlenecks, and endurance testing to ensure system stability over extended periods. Performance testing helps uncover performance issues, such as slow response times, resource utilization problems, or scalability concerns.

Security Testing: Security testing is crucial to identify vulnerabilities and ensure that sensitive user data remains protected. This involves testing for potential security risks, such as SQL injection, cross-site scripting (XSS), authentication and authorization flaws, and data encryption. Security testing helps validate the effectiveness of implemented security measures, ensuring the system's resilience against unauthorized access, data breaches, or other security threats.

Usability Testing: Usability testing focuses on evaluating the system's user interface (UI) and user experience (UX). It involves testing the system with representative end-users to identify any usability issues, navigational difficulties, or design flaws. Usability testing helps ensure that the system is intuitive, user-friendly, and meets the needs and expectations of its target users.

Test Cases :

Test Case No.	Description	Input	Expected Output	Actual Output	Result
TC001	Ticket Booking - Valid booking request	Selected train, journey date, passenger details	Confirmation message with ticket details	Confirmation message with ticket details	Pass
TC002	Ticket Booking - Invalid train selection	Invalid train number	Error message "Invalid train selection"	Error message "Invalid train selection"	Pass
TC003	Train Searching -	Source and	List of available	List of available	Pass

Test Case No.	Description	Input	Expected Output	Actual Output	Result
	Valid search	destination stations	trains matching criteria	trains matching criteria	
TC004	Train Searching - Invalid station input	Invalid station code	Error message "Invalid station code"	Error message "Invalid station code"	Pass
TC005	User Login - Valid credentials	Username and password	Redirect to user profile page	Redirect to user profile page	Pass
TC006	User Login - Invalid credentials	Incorrect username or password	Error message "Invalid username or password"	Error message "Invalid username or password"	Pass
TC007	Profile Access - View user profile	User selects "View Profile" option	User details and reservation history shown	User details and reservation history shown	Pass
TC008	Profile Update - Valid update	Updated user details	Success message "Profile updated successfully"	Success message "Profile updated successfully"	Pass
TC009	PNR Status Checking - Valid PNR	PNR number	PNR status details	PNR status details	Pass
TC010	PNR Status Checking - Invalid PNR	Invalid PNR number	Error message "Invalid PNR number"	Error message "Invalid PNR number"	Pass
TC011	Ticket Cancellation - Valid cancellation	Ticket number	Success message "Ticket canceled successfully"	Success message "Ticket canceled successfully"	Pass
TC012	Ticket Cancellation - Invalid ticket number	Invalid ticket number	Error message "Invalid ticket number"	Error message "Invalid ticket number"	Pass
TC013	Train Status - View train status	Selected train	Train status details	Train status details	Pass
TC014	History of Reservations - View reservation details	Selected reservation	Reservation details displayed	Reservation details displayed	Pass
TC015	Email and OTP Verification - Successful verification	Entered valid email and OTP	Email and OTP verified successfully	Email and OTP verified successfully	Pass
TC016	Email and OTP Verification - Invalid email or OTP	Incorrect email or OTP	Error message "Invalid email or OTP"	Error message "Invalid email or OTP"	Pass

5. System Design

5.1 Architecture Design

The architecture of the railway ticket booking system is designed using a client-server model with multiple layers, including the Presentation Layer (frontend), Application Layer (backend/API), Business Logic Layer, and Data Layer (database). The tech stack comprises React.js, Node.js, Express.js, and MongoDB. In the Presentation Layer, React.js is used to handle the user interface, collect user input, and display information. The Application Layer, built with Node.js and Express.js, manages the routing, request processing, and response handling. It also implements authentication and middleware for various tasks. The Business Logic Layer contains the core logic and rules of the application. It handles functionalities such as ticket booking, user management, and train addition. The Data Layer utilizes MongoDB and Mongoose to store and manage persistent data, interacting with the database for data retrieval and updates. The flow of communication starts with the frontend sending HTTP requests to the backend API, which handles authentication and routes requests to the appropriate handlers. The Business Logic Layer processes the requests, enforces business rules, and interacts with the Data Layer for data retrieval and updates. The backend sends back HTTP responses to the frontend, which updates the user interface accordingly. This architecture enables seamless user interactions, secure authentication, efficient data processing, and reliable data storage. It leverages the strengths of each layer and the capabilities of the tech stack to deliver a robust and scalable railway ticket booking system.

More detailed and step-wise explanation is provided below:

1. Presentation Layer (Frontend):

- Tech Stack: React.js, HTML, CSS
- Responsible for the user interface and user interaction.
- Renders the UI components, collects user input, and displays information.
- Manages the user flow and navigation within the application.
- Sends HTTP requests to the backend API for data processing and retrieval.
- Receives and displays responses from the backend.
- Implements client-side form validation and input sanitization.

2. Application Layer (Backend/API):

- Tech Stack: Node.js, Express.js
- Handles the routing, request processing, and response handling.
- Receives HTTP requests from the frontend and routes them to appropriate handlers.
- Implements middleware for request processing tasks like authentication, logging, error handling, etc.

- Decodes and verifies JSON Web Tokens (JWT) to authenticate and authorize users.
- Communicates with the Business Logic Layer for data processing and business rules.
- Orchestrates the flow of data and services between layers.
- Sends HTTP responses back to the frontend with appropriate status codes and data.

3. Business Logic Layer:

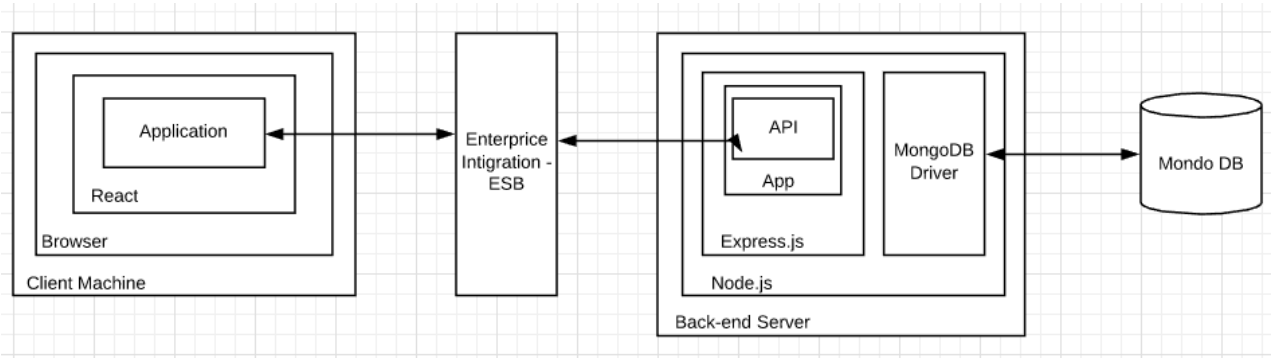
- Tech Stack: Node.js
- Contains the core logic and rules of the application.
- Handles the business workflows and operations for each functionality.
- Implements the business rules, validations, and calculations.
- Coordinates with the Data Layer for data retrieval and updates.
- Performs operations such as ticket booking, user management, train addition, etc.
- Enforces business-specific constraints, like seat availability, fare calculations, etc.
- May interact with external services or APIs for additional functionality, such as email notifications or OTP verification.

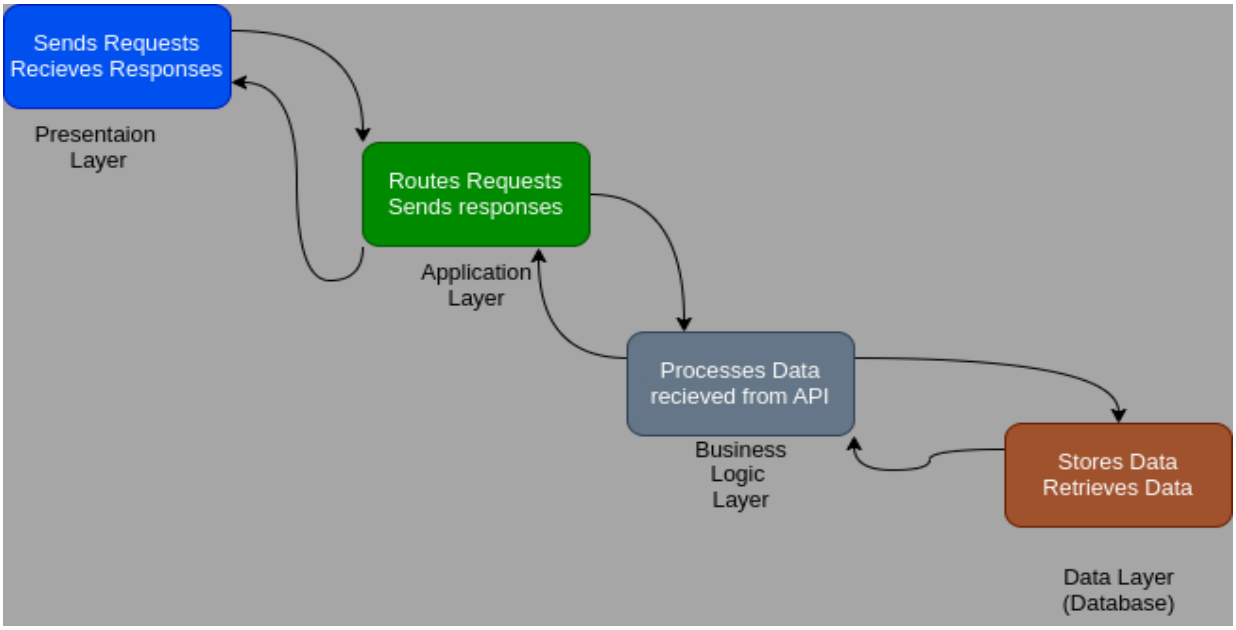
4. Data Layer (Database):

- Tech Stack: MongoDB, Mongoose
- Stores and manages the application's persistent data.
- Utilizes MongoDB as the database system.
- Defines data models and schemas using Mongoose.
- Handles data retrieval, storage, and updates.
- Implements CRUD (Create, Read, Update, Delete) operations for the relevant entities (users, tickets, trains, etc.).

The flow of communication between the layers:

The Presentation Layer (frontend) collects user input and sends HTTP requests to specific endpoints defined in the Application Layer (backend). The Application Layer receives the requests, applies middleware for tasks like authentication, and routes them to appropriate handlers based on the endpoints. The Business Logic Layer receives the requests and performs the necessary operations specific to each functionality. It enforces business rules, validates inputs, and interacts with the Data Layer as needed. The Data Layer handles data retrieval and storage using MongoDB. It communicates with the database to execute queries, retrieve or update data, and ensures data integrity. The Business Logic Layer processes the data retrieved from the Data Layer, performs required calculations, transformations, or validations, and prepares the response data. The Application Layer receives the response from the Business Logic Layer and sends back an HTTP response to the Presentation Layer (frontend) with the appropriate data and status codes. The Presentation Layer (frontend) receives the response, updates the user interface, and presents the relevant information or prompts to the user.





Overall System architecture

5.2 Database Design

Database design refers to the process of structuring and organizing data in a database management system (DBMS) to meet specific requirements and optimize data storage and retrieval. It involves defining the structure, relationships, and constraints of the data, as well as determining the appropriate data types and indexing strategies.

In the context of the railway ticket booking system, the database design for MongoDB would involve designing the schema and collections to store data related to users, tickets, trains, stations, and other relevant entities. Considerations for the database design are :

1. Entities: Entities that need to be stored in the database, such as users, tickets, trains, stations, etc.

2. Relationships: Identify the relationships between entities, such as the association between users and tickets, trains and stations, etc. This helps establish how the data is interconnected, although the entities are related but the tech stack used for this project is non-relational database so there is no entity relation diagram except the tables.

3. Collections: Separate collections for each entity to store related data.

4. Schema: Structure of each collection by specifying the fields and their data types.

5. Optimization for performance: Considering the system's performance requirements and design the database to efficiently handle data operations, such as read and write operations, while minimizing resource usage.

Structure of the tables for each of the Entities are provided below.

5.4 Data Flow Diagrams

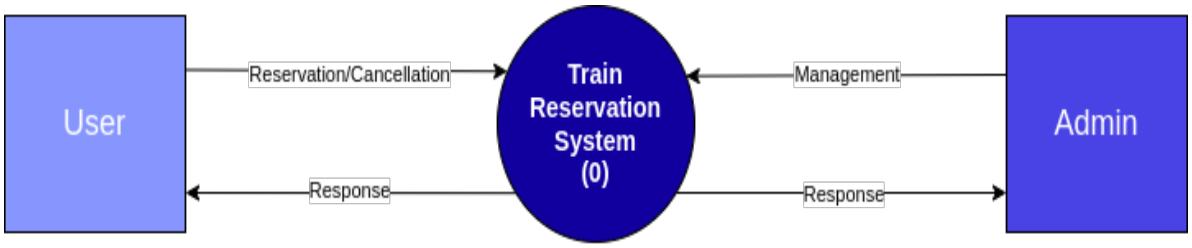
Data Flow Diagrams:

A Data Flow Diagram (DFD) is a graphic depiction that shows how data moves through a system. The activities, data stores, data flows, and external entities involved in a system are represented using this modelling technique, which is utilized in software engineering and system analysis.

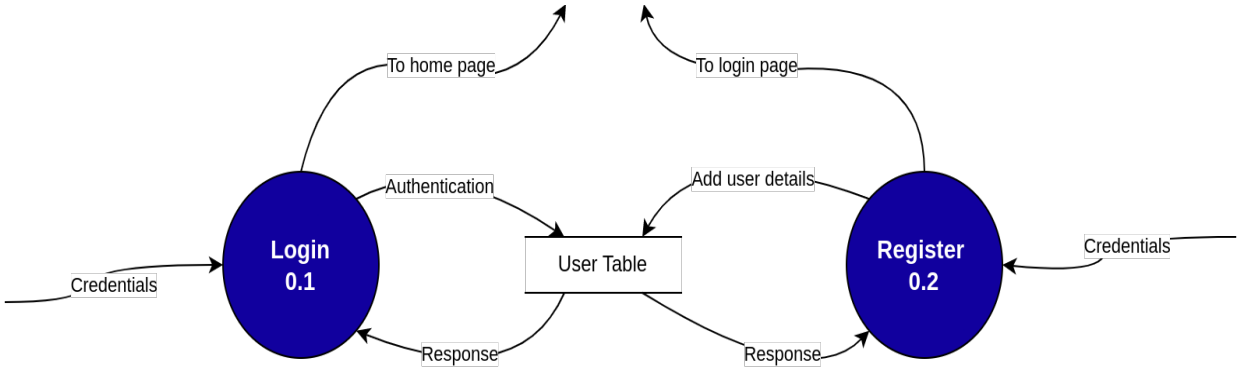
DFDs are available in a variety of levels of complexity, from the high-level (Level-0), which illustrates how the system interacts with its external entities, to more in-depth levels, which deconstruct operations into smaller steps and outline data flows within the system.

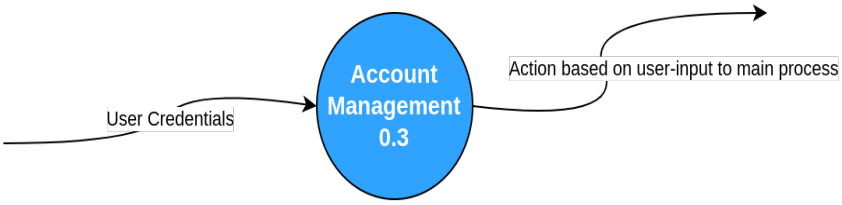
System analysts, stakeholders, and designers can better understand, document, and improve system functionality and data flow by using DFDs as a communication and analytical tool.

Level – 0 DFD

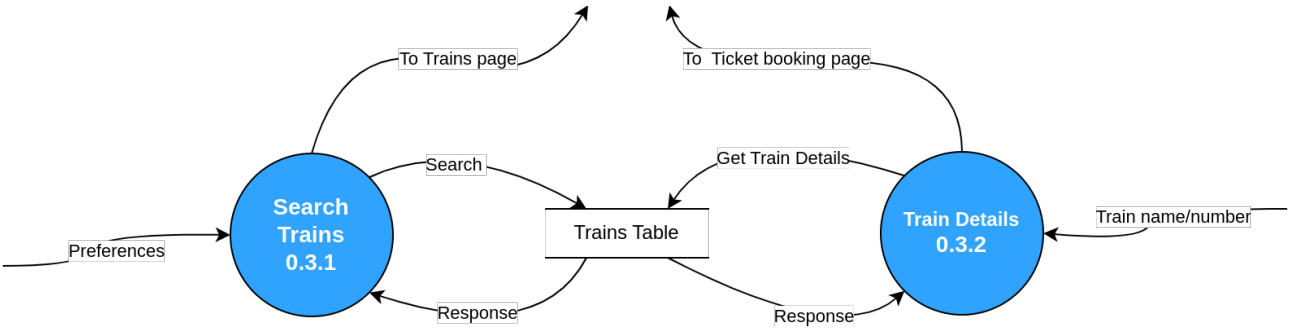


Level –1 DFD

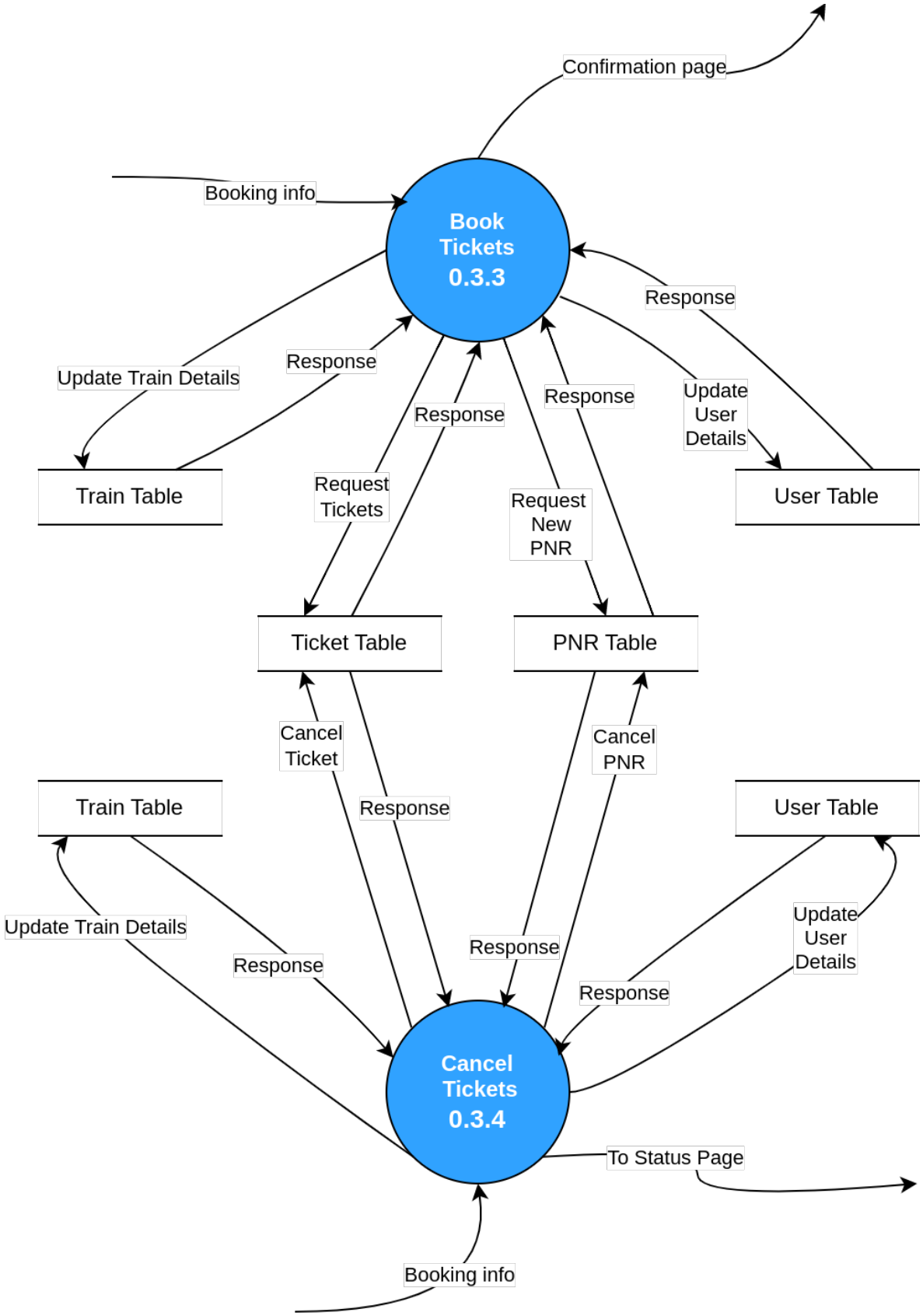




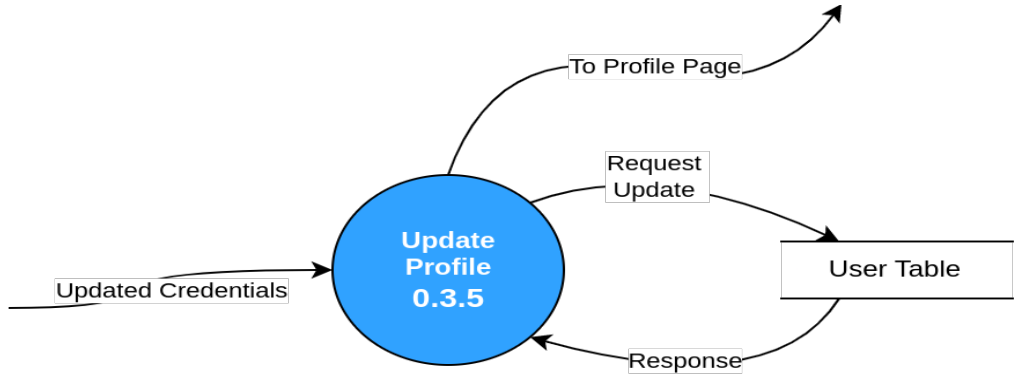
Level –2 DFD

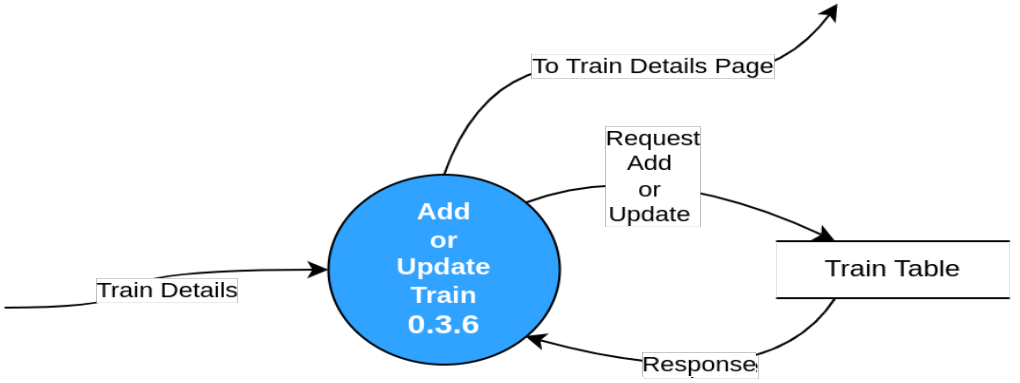


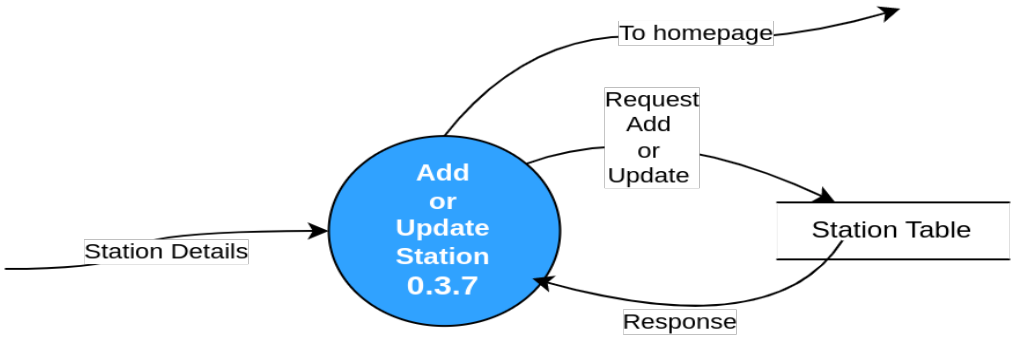
Level – 2 DFD

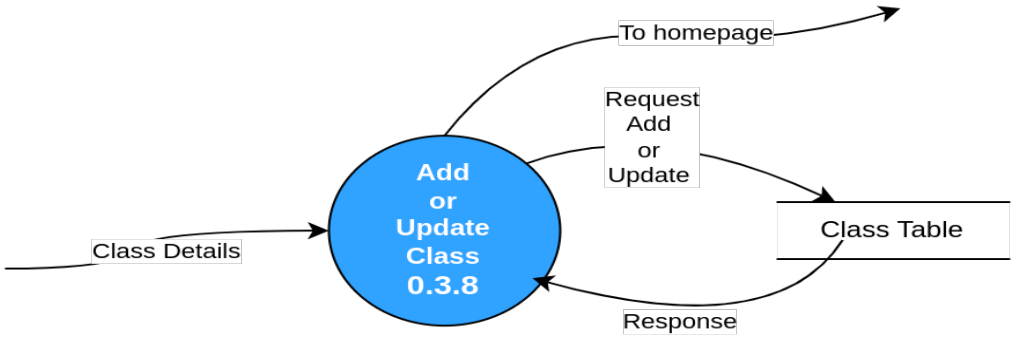

Confirmation page
Booking info
Book Tickets 0.3.3
Response
Update Train Details
Response
Response
Response
Update User Details
Train Table
Request Tickets
Request New PNR
User Table
Ticket Table
PNR Table
Cancel Ticket
Cancel PNR
Train Table
Response
User Table
Update Train Details
Response
Response
Update User Details
Response
Cancel Tickets 0.3.4
To Status Page
Booking info

Level – 2 DFD


To Profile Page
Request Update
Update Profile 0.3.5
User Table
Updated Credentials
Response


To Train Details Page
Request Add or Update
Add or Update Train 0.3.6
Train Table
Train Details
Response


To homepage
Request Add or Update
Add or Update Station 0.3.7
Station Table
Station Details
Response


To homepage
Request Add or Update
Add or Update Class 0.3.8
Class Table
Class Details
Response

5.5 Algorithm Design

The algorithm design for the railway ticket reservation system encompasses various functionalities and operations. The user registration algorithm involves hashing the password using a secure hashing algorithm such as bcrypt to protect user data. The hashed password is stored in the MongoDB database along with other user details. For user login, the algorithm retrieves user details from the MongoDB database based on the provided email and compares the stored hashed password with the provided password using a secure password comparison algorithm. If the passwords match, a session token is generated for authenticated access.

To search for trains, the algorithm queries the MongoDB database using the source and destination stations. Indexing can be used on the source and destination station fields to optimize the search performance. The search complexity, with proper indexing, can be reduced to O(log n) or O(1), depending on the size of the database.

When booking tickets, the algorithm checks the availability of seats for the selected train and class. This may involve querying the database to determine the number of available seats. If seats are available, the algorithm reserves the required number of seats and generates a unique booking ID, such as a UUID. The booking details are stored in the MongoDB database. The complexity of the booking process depends on factors such as the number of seats and the availability check algorithm. With efficient indexing and availability checks, the complexity can be reduced to O(log n) or O(1).

OTP verification is performed by generating a random OTP using a secure random number generator. The OTP is then sent to the user's registered email or phone number using a messaging service. The user is prompted to enter the received OTP for verification, and the entered OTP is validated against the generated OTP. The complexity of OTP generation and validation is typically O(1).

For payment processing, the algorithm collects payment details from the user and integrates with a payment gateway to securely process the payment. The payment status is validated and confirmed. The complexity of payment processing depends on the chosen payment gateway and the associated payment processing algorithms.

Additionally, the algorithm design includes functionalities to add trains, stations, and classes. When adding trains, a unique train ID, such as a UUID, is generated, and the train details are stored in the MongoDB database. Similarly, unique IDs are generated for adding stations and classes, and their details are stored in the database. The complexities for adding trains, stations, and classes are typically O(1).

Profile viewing involves retrieving the user's profile details from the MongoDB database based on the user ID and returning them to the user interface. With proper indexing, the complexity can be reduced to O(log n) or O(1). Profile updating includes validating and sanitizing the provided user details, such as name, email, and password, and updating the user's profile in the database. The complexity for profile updating is typically O(1).

Furthermore, MongoDB is utilized for data storage, employing BSON (Binary JSON), a JSON-like document data model. The database uses a B-tree data structure (BSON-tree) for indexing and efficient retrieval of data. B-trees provide logarithmic time complexity (O(log n)) for search, insertion, and deletion operations. MongoDB's flexible document model allows for efficient storage and retrieval of complex data structures without the need for complex JOIN operations seen in relational databases.

The complexities mentioned are theoretical approximations and may vary in practice based on factors such as database size, indexing strategies, query optimization, and hardware capabilities.

5.6 System Maintenance

System maintenance involves various tasks to ensure the smooth operation and reliability of the railway ticket reservation system. Regular database maintenance activities are crucial for optimal system performance. This includes routine backups of the MongoDB database to prevent data loss in case of hardware failures or other unforeseen events. Additionally, database indexes should be periodically monitored and optimized to enhance query performance and minimize response times for search operations.

To ensure data integrity and security, the system should implement regular security audits and vulnerability assessments. This involves conducting periodic security scans, identifying potential vulnerabilities, and applying necessary patches and updates to the system components, including the MongoDB database, Node.js backend, and React.js frontend. Regular security audits also help identify and address any potential risks related to user authentication, authorization, and data protection.

Another important aspect of system maintenance is monitoring and performance optimization. It is crucial to monitor system resources such as CPU, memory, and network utilization to identify any bottlenecks or performance issues. Monitoring tools can help track system metrics, detect anomalies, and provide insights into system health and resource utilization patterns. This data can be used to optimize the system's performance by identifying and resolving any performance bottlenecks, optimizing database queries, and improving response times.

Furthermore, the system should have proper error handling mechanisms in place. Comprehensive logging and error tracking should be implemented to capture and analyze system errors, exceptions, and failures. This allows for proactive identification and resolution of issues, improving the system's stability and user experience.

Regular testing and quality assurance are vital for system maintenance. This includes conducting periodic functional tests, integration tests, and performance tests to ensure that all functionalities are working as expected and that the system can handle the expected load. Automated testing frameworks can help automate the testing process and provide timely feedback on any issues or regressions.

In summary, system maintenance for the railway ticket reservation system involves activities such as database maintenance, security audits, monitoring and performance optimization, error handling, and testing. By regularly performing these tasks, the system can maintain high availability, data integrity, security, and performance, ensuring a seamless experience for users and minimizing disruptions to the ticket reservation process.

5.7 System Development Approach

The system development approach for the railway ticket reservation project follows the iterative waterfall model, which combines the principles of the traditional waterfall model with iterative and incremental development. The process begins with requirements gathering, where the development team conducts interviews with stakeholders and analyzes existing systems to gather and document the system requirements. This phase aims to identify the functionalities, user interfaces, performance expectations, and security considerations for the ticket reservation system. The gathered requirements are then documented in a detailed specification.

Once the requirements are gathered, the system design phase begins. This phase involves defining the system architecture and design based on the gathered requirements. The development team plans the architectural components, database schema, and interfaces between different modules of the system. Additionally, the user interface layout and workflows are designed and documented during this phase.

After the system design phase, the implementation phase begins. The development team starts coding and developing the system based on the design specifications. The backend of the system is developed using Node.js and MongoDB for data storage, while the frontend is developed using React.js for creating user interfaces. Each iteration of the implementation phase focuses on implementing a subset of functionalities based on their priority and importance.

The iterative development phase is a key aspect of the iterative waterfall model. After the initial implementation, the project moves into an iterative development phase, where each iteration follows a mini-waterfall process. Each iteration starts with requirements analysis, followed by system design, implementation, testing, and deployment of a working subset of functionalities. The iterations are typically short and timeboxed, lasting a few weeks. The cycle repeats until all major functionalities are implemented.

Testing is an integral part of each iteration and is conducted during the iterative development phase. Various types of testing, such as unit testing, integration testing, functional testing, and performance testing, are performed to ensure that the developed functionalities meet the specified requirements and perform as expected. Defects and issues identified during testing are logged, tracked, and resolved in subsequent iterations.

Once an iteration is completed, the tested and approved functionalities are deployed to a staging environment. The staging environment allows for final user acceptance testing (UAT) and validation. Feedback from stakeholders and end users is collected during this phase, and it is used to refine the system and incorporate necessary changes and enhancements in subsequent iterations.

After incorporating feedback and making necessary refinements, the final testing phase is conducted to ensure that the system meets all requirements and is ready for production. Once the system passes the final testing phase, it is deployed to the live environment and made available to end users.

Following deployment, the system enters the maintenance and support phase. This phase involves monitoring the system, addressing any issues or bugs reported by users, and providing ongoing support. Regular maintenance activities, such as updates, bug fixes, and enhancements, are performed as needed to ensure the system's reliability and effectiveness.

Throughout the iterative waterfall model, documentation is crucial at every stage of the development process. Requirements documentation, design specifications, test plans, and user manuals are maintained to ensure a clear understanding of the system and facilitate smooth collaboration among the development team, stakeholders, and end users.

By adopting an iterative waterfall model, the railway ticket reservation project benefits from a structured and well-defined approach, while allowing for flexibility, user feedback, and continuous improvement throughout the development process.

6. System Implementation

6.1 System Development Platform

I used my personal workstation for this system development project. It had an Intel i5 11th generation processor, 8 GB RAM, and 512 GB storage. The workstation ran on Ubuntu 22.04, providing a stable and reliable operating system for development tasks. With its powerful processor and sufficient memory, the workstation ensured smooth and efficient performance during the development process. It provided ample storage space for the project files, libraries, and dependencies required for the development environment.

The development environment for this project was set up on Ubuntu, which offered a robust and user-friendly platform for software development. I utilized Sublime as my text editor, providing a lightweight yet feature-rich code editing experience. The Linux terminal was my go-to tool for executing server-related commands and managing the project's server environment. Node.js was installed on the workstation, allowing me to leverage its runtime environment for building server-side applications. I also utilized Nodemon, a development tool, to automatically restart the server whenever code changes were detected. Furthermore, I employed GitHub for version control, enabling easy collaboration, code sharing, and tracking changes throughout the development process.

On my Ubuntu workstation, I set up a development environment using the aforementioned tools and technologies. I installed and configured Node.js, which served as the runtime environment for executing JavaScript code on the server-side. Additionally, I utilized the Express framework, a popular choice for building web applications with Node.js, to create a robust and scalable backend. MongoDB, a NoSQL database, was integrated into the development environment to store and

retrieve data. It offered flexibility and scalability, making it suitable for handling the system's data requirements. To streamline the development process, I utilized Nodemon to automatically restart the server on code changes, allowing for faster testing and debugging.

For this project, I opted to use MongoDB as the database. MongoDB is a NoSQL document database that provides a flexible and scalable solution for storing and managing data. It employs a JSON-like document structure, allowing for the storage of data in a flexible and schema-less manner. MongoDB offers features such as automatic sharding, replication, and horizontal scalability, making it suitable for handling large amounts of data and high traffic loads. Its query language and indexing capabilities enable efficient data retrieval and search operations. With MongoDB, I implemented a database schema that aligned with the system's data model, allowing for efficient storage and retrieval of data.

The system development involved utilizing a tech stack comprising HTML, CSS, Tailwind CSS, React.js, Node.js, Express, and MongoDB. HTML and CSS were used for creating the system's user interface, defining the structure and styling of the web pages. Tailwind CSS, a utility-first CSS framework, provided a streamlined approach to styling, allowing for rapid development. React.js, a JavaScript library, facilitated the creation of interactive and dynamic user interfaces, enhancing the system's user experience. Node.js served as the runtime environment for executing server-side JavaScript code, while Express provided a robust framework for building the system's backend. MongoDB, a NoSQL database, was integrated into the stack for efficient data storage and retrieval. This tech stack provided a comprehensive and efficient solution for developing the system, combining the power of modern front-end and back-end technologies.

6.2 User intefaces

6.3 Database

7. Data Dictionary